

Hardening the Kernel

A Strategic Guide to Application Sandboxing and Syscall Isolation with seccomp

STRICTLY FOR ENGINEERING TEAMS



The Attack Surface Problem

Why System Calls are the Gateway to Kernel Compromise

The Syscall Gateway

System calls represent the fundamental interface between Userland and the Kernel. While essential, they provide an expansive vector for exploitation.



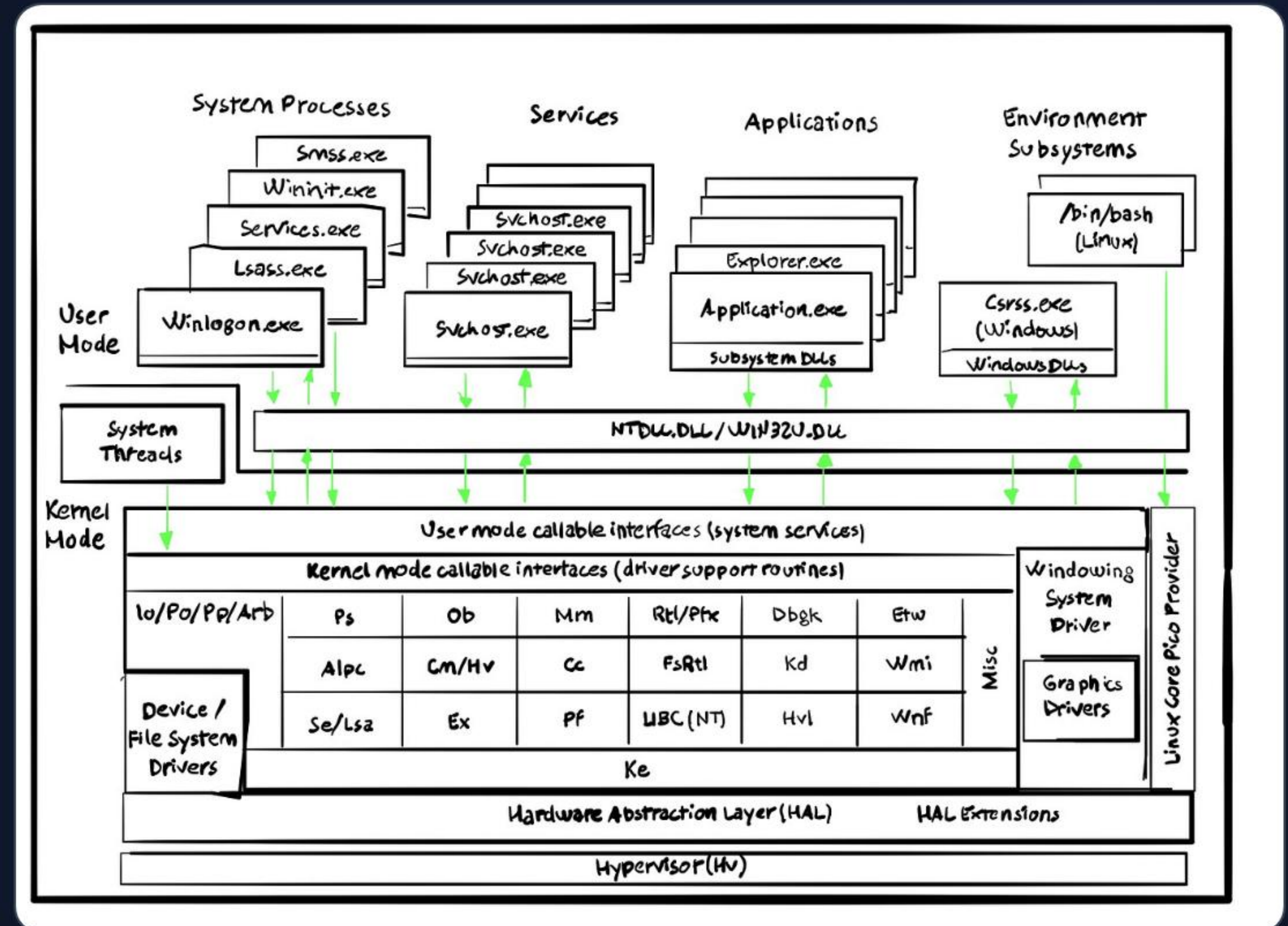
Modern kernels expose 450+ syscalls



Most apps use less than 10% of them



Untrusted syscalls increase privilege escalation risk



The Core Pillars of seccomp



Isolation

Reducing reachable code paths within the kernel, ensuring a vulnerability cannot be reached.



Least Privilege

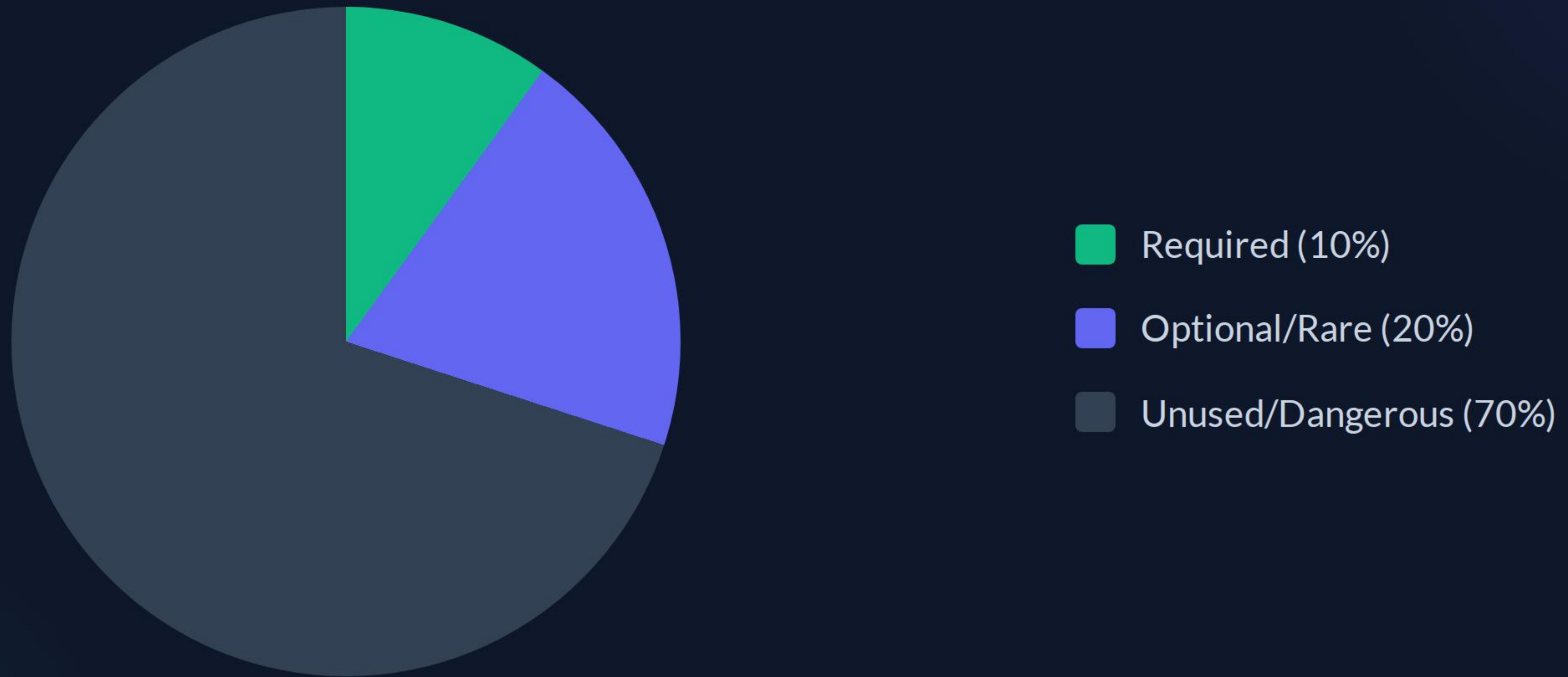
Ensuring an application only has the exact capabilities required for its functional scope.



Predictability

Enforcing immutable behavior on application execution threads at the hardware/software boundary.

| Syscall Utilization Analysis



Attack surface reduction is achieved by blocking the 70% of "Dark Matter" syscalls.

| The Evolution of seccomp



2005

Mode 1 Merged
(Strict Mode)



2012

Mode 2 (BPF)
Filtering Added



2014

seccomp()
System Call



2025

Modern Embedded
Maturity

Comparing Operational Modes

Mode 1: Strict

- The Original Security Profile
- Allows only read, write, exit, sigreturn
- Immediate SIGKILL on violation
- Immutable and High-Performance

Mode 2: Filter

- BPF-powered flexibility
- Custom Allow/Deny lists
- Argument-level inspection
- The modern standard for app security

Growth of Kernel Syscall Complexity



Total Number of Available System Calls (x86_64 Architecture)

| The libseccomp Advantage

-  **Declarative API:** Avoid writing raw BPF instructions by hand.
-  **Architecture Independence:** Policy remains portable across x86, ARM, and RISC-V.
-  **Easier Debugging:** Transparent error reporting and logging profiles.
-  **Safe Defaults:** Automatically implements "default-deny" logic securely.

Real-World Adoption



Docker

Default seccomp profile blocks ~44 syscalls (swapon, reboot, etc.)



Chrome

Heavy use of seccomp to isolate render processes from host kernel.



OpenSSH

Uses seccomp to sandbox the pre-auth login process.

Critical Metrics

450+

Total modern syscalls

Scale of Exposure

Every single one of these 450+ entry points is a potential vulnerability if not properly isolated. Your goal today is to reduce this number to the absolute minimum for your applications.

Ready for the Lab?

Next Steps: Buildroot, QEMU, and Isolation Debugging

Questions before we begin the Hands-on Assignment?



LAB STARTS NOW

Image Sources



https://miro.medium.com/v2/resize:fit:1400/0*_wVoFdo8cbczvAll.png

Source: medium.com



<https://uxwing.com/wp-content/themes/uxwing/download/brands-and-social-media/docker-icon.svg>

Source: uxwing.com



<https://cdn3d.iconscout.com/3d/free/preview/free-google-chrome-3d-icon-png-download-2447913.png?f=webp&h=700>

Source: iconscout.com



https://static.vecteezy.com/system/resources/previews/032/316/950/non_2x/ssh-secure-shell-doodle-icon-design-illustration-networking-symbol-on-white-background-eps-10-file-vector.jpg

Source: www.vecteezy.com